# Two Refusals, One Ledger: No-Cloning and No-Deleting as Occurrence Conservation, with the Bills

Working paper v0.1, Project secret3, August 11, 2026

## Abstract

Quantum mechanics forbids copying an unknown state (no cloning, Wootters and Zurek 1982, Dieks 1982) and forbids deleting one (no deleting, Pati and Braunstein 2000), and it is well understood that the structural shadow of both prohibitions is linearity: in the linear-logical axiomatization the two theorems just are the absence of the diagonal and of the projection. This paper compiles that shadow for the two-move calculus, in a formulation that needs no fragment predicate at all. A per-step conservation law, occStepEq, states that the occurrence count of any watched name changes only through two metered events: the K rule may erase it, the S rule may duplicate it, and nothing else touches it. Summed along any run (occRunEq), the law yields the two refusals as corollaries: a computation whose duplication meter reads zero cannot turn one occurrence into two (noClone), and one whose erasure meter reads zero cannot turn one occurrence into none (noDelete), quantified over all terms, all reduction sequences, all strategies. Two further corollaries state the prices: turning one into two costs at least one metered share (cloneBill), and turning one into none costs at least one metered drop (deleteBill). The bills connect the refusals to the calculus's thermodynamic ledger: every deletion of a watched payload is a K event of the kind the entropy budget prices in Landauer's currency. The paper closes with an honest scope note: these are structural cousins of the quantum theorems, not Hilbert-space results, and the value claimed is the mechanization, the fragment-free formulation, and the quantitative bills.

## 1. The shadow, stated honestly

There is a standard bridge between quantum impossibilities and resource logic. On the categorical side, Abramsky's treatment of no-cloning shows that admitting a copying diagonal into a compact closed setting collapses the dynamics, every endomorphism a scalar multiple of the identity, the physical twin of logic's explosion, one contradiction proving everything. On the calculus side, linear languages exclude the cloning map by typing. What this paper adds is small and precise: the untyped, ledger-based combinatory version, machine-checked, with the prohibition and its price in the same theorem set. No linearity predicate is defined and none is needed; instead of fencing a fragment, the law meters the whole calculus and lets the meters do the fencing.

## 2. The conservation law

For a watched name x, define per step the erased amount eOcc (the occurrence count of x in the subtree that RedK discards; zero elsewhere) and the duplicated amount dOcc (the count in the argument that RedS shares; zero elsewhere), each threaded through contexts. The law:

```
occStepEq : occ x u + eOcc x s = occ x t + dOcc x s
```

what arrives plus what was erased equals what departed plus what was duplicated, proved by one induction with two small addition juggles. Summing the meters along a run gives eOccs and dOccs and the run-level law occRunEq of the same shape. The law is exhaustive by construction: the two

rewrite rules are the only clauses of the step relation that move subtrees, so the two meters are provably the only doors through which an occurrence can leave or multiply.

### 3. The refusals and the bills

```
noClone    : occ x t = 1 -> dOccs x ss = 0 -> Not (occ x u = 2)
noDelete   : occ x t = 1 -> eOccs x ss = 0 -> Not (occ x u = 0)
cloneBill  : occ x t = 1 -> occ x u = 2 -> LTE 1 (dOccs x ss)
deleteBill : occ x t = 1 -> occ x u = 0 -> LTE 1 (eOccs x ss)
```

Read aloud: the calculus refuses to clone what it never shares and refuses to delete what it never drops; and when a computation does clone or delete, the respective meter has provably clicked at least once. The deletion bill is the point of contact with the series' thermodynamics: the metered K events are exactly the erasures whose sizes the entropy budget paper sums and prices at Landauer's floor, so the structural refusal and the physical bill are two readings of one ledger. In the linear fragment, where by construction neither meter ever clicks, both refusals hold unconditionally; the fragment-free formulation is strictly stronger, since it also governs every computation outside the fragment, with the meters recording the exceptions.

### 4. Scope, and the two literatures

What this is not: a theorem about Hilbert spaces, fidelities, or unitarity; the quantum no-cloning theorem's content (only identical or orthogonal states admit a common cloner) has no analogue here, and the Isabelle formalization of that statement is a different kind of object. What it is: the combinatory instance of the linearity shadow, in the calculus this program owns, with the smallest possible trusted base and a quantitative extension the typed treatments do not state. The two literatures it sits between: categorical quantum mechanics (Abramsky, Coecke) and linear combinatory algebras (Abramsky, Haghverdi, Scott) on one side; the quantitative rewriting ledger of this series on the other.

### Appendix A: Methods

All theorems are in the shared proof module Proofs.idr, 2,890 lines after this paper's additions, %default total, no postulates, no holes; idris2 —check Proofs.idr is the entire verification.

### Appendix B: The ledger section (module part V)

```
--------------------------------------------------------------------
-- Tier 2, part V: two refusals, one ledger (occurrence conservation)
--------------------------------------------------------------------

||| what a single step erases of the watched name: only RedK drops
eOcc : (x : String) -> {t, u : Term} -> Step t u -> Nat
eOcc x (RedK {a} {b}) = occ x b
eOcc x (RedS {a} {b} {c}) = 0
eOcc x (CtxL s) = eOcc x s
eOcc x (CtxR s) = eOcc x s

||| what a single step duplicates of the watched name: only RedS shares
dOcc : (x : String) -> {t, u : Term} -> Step t u -> Nat
dOcc x (RedK {a} {b}) = 0
dOcc x (RedS {a} {b} {c}) = occ x c
dOcc x (CtxL s) = dOcc x s
dOcc x (CtxR s) = dOcc x s
```

```
shuffleN : (a, b, c : Nat) -> (a + c) + (b + c) = ((a + b) + c) + c
shuffleN Z b c = plusCommutative c (b + c)
shuffleN (S a) b c = cong S (shuffleN a b c)

juggleL : {p, q, r, e, d : Nat} -> p + e = q + d ->
  (p + r) + e = (q + r) + d
juggleL prf =
  trans (sym (plusAssociative p r e)) $
  trans (cong (p +) (plusCommutative r e)) $
  trans (plusAssociative p e r) $
  trans (cong (+ r) prf) $
  trans (sym (plusAssociative q d r)) $
  trans (cong (q +) (plusCommutative d r)) $
  plusAssociative q r d

juggleR : {p, q, l, e, d : Nat} -> p + e = q + d ->
  (l + p) + e = (l + q) + d
juggleR prf =
  trans (sym (plusAssociative l p e)) $
  trans (cong (l +) prf) $
  plusAssociative l q d

||| the conservation law, per step: what arrives plus what was erased
||| equals what departed plus what was duplicated
occStepEq : (x : String) -> {t, u : Term} -> (s : Step t u) ->
  occ x u + eOcc x s = occ x t + dOcc x s
occStepEq x (RedK {a} {b}) =
  rewrite plusZeroRightNeutral (occ x a + occ x b) in Refl
occStepEq x (RedS {a} {b} {c}) =
  rewrite plusZeroRightNeutral
    ((occ x a + occ x c) + (occ x b + occ x c)) in
  shuffleN (occ x a) (occ x b) (occ x c)
occStepEq x (CtxL {r} s) = juggleL (occStepEq x s)
occStepEq x (CtxR {l} s) = juggleR (occStepEq x s)

||| the meters along a whole run
eOccs : (x : String) -> {t, u : Term} -> Steps t u -> Nat
eOccs x Done = 0
eOccs x (Then s ss) = eOcc x s + eOccs x ss

dOccs : (x : String) -> {t, u : Term} -> Steps t u -> Nat
dOccs x Done = 0
dOccs x (Then s ss) = dOcc x s + dOccs x ss

||| the conservation law, along any run
occRunEq : (x : String) -> {t, u : Term} -> (ss : Steps t u) ->
  occ x u + eOccs x ss = occ x t + dOccs x ss
occRunEq x Done = Refl
occRunEq x (Then {a} {b} s ss) =
  trans (cong ((occ x u) +) (plusCommutative (eOcc x s) (eOccs x ss))) $
  trans (plusAssociative (occ x u) (eOccs x ss) (eOcc x s)) $
  trans (cong (+ (eOcc x s)) (occRunEq x ss)) $
  trans (sym (plusAssociative (occ x b) (dOccs x ss) (eOcc x s))) $
  trans (cong ((occ x b) +) (plusCommutative (dOccs x ss) (eOcc x s))) $
```

```
  trans (plusAssociative (occ x b) (eOcc x s) (dOccs x ss)) $
  trans (cong (+ (dOccs x ss)) (occStepEq x s)) $
  trans (sym (plusAssociative (occ x a) (dOcc x s) (dOccs x ss))) $
  Refl

twoNotOne : (e : Nat) -> S (S e) = 1 -> Void
twoNotOne e Refl impossible

onePlusNotZero : (d : Nat) -> 1 + d = 0 -> Void
onePlusNotZero d Refl impossible

||| no cloning without sharing: a run whose duplication meter reads
||| zero cannot turn one occurrence of the watched name into two
noClone : (x : String) -> {t, u : Term} -> (ss : Steps t u) ->
  occ x t = 1 -> dOccs x ss = 0 -> Not (occ x u = 2)
noClone x ss pt pd pu =
  twoNotOne (eOccs x ss) $
    trans (cong (+ (eOccs x ss)) (sym pu)) $
    trans (occRunEq x ss) $
    trans (cong2 (+) pt pd) $
    plusZeroRightNeutral 1

||| no deleting without dropping: a run whose erasure meter reads
||| zero cannot turn one occurrence of the watched name into none
noDelete : (x : String) -> {t, u : Term} -> (ss : Steps t u) ->
  occ x t = 1 -> eOccs x ss = 0 -> Not (occ x u = 0)
noDelete x ss pt pe pu =
  onePlusNotZero (dOccs x ss) $
    trans (cong (+ (dOccs x ss)) (sym pt)) $
    trans (sym (occRunEq x ss)) $
    trans (cong2 (+) pu pe) $
    Refl

twoEqOnePlus : (e, d : Nat) -> S (S e) = S d -> d = S e
twoEqOnePlus e d Refl = Refl

zeroPlusOne : (e, d : Nat) -> e = S d -> LTE 1 e
zeroPlusOne e d prf = rewrite prf in LTESucc LTEZero

||| the cloning bill: turning one into two costs at least one share
cloneBill : (x : String) -> {t, u : Term} -> (ss : Steps t u) ->
  occ x t = 1 -> occ x u = 2 -> LTE 1 (dOccs x ss)
cloneBill x ss pt pu =
  let h : (S (S (eOccs x ss)) = S (dOccs x ss))
        := trans (cong (+ (eOccs x ss)) (sym pu)) $
           trans (occRunEq x ss) $
           cong (+ (dOccs x ss)) pt
  in rewrite twoEqOnePlus (eOccs x ss) (dOccs x ss) h in
     LTESucc LTEZero

oneEqZeroPlus : (d, e : Nat) -> S d = e -> LTE 1 e
oneEqZeroPlus d e prf = rewrite sym prf in LTESucc LTEZero

||| the deletion bill: turning one into none costs at least one drop
deleteBill : (x : String) -> {t, u : Term} -> (ss : Steps t u) ->
```

```
  occ x t = 1 -> occ x u = 0 -> LTE 1 (eOccs x ss)
deleteBill x ss pt pu =
  let h : (S (dOccs x ss) = eOccs x ss)
        := trans (cong (+ (dOccs x ss)) (sym pt)) $
           trans (sym (occRunEq x ss)) $
           cong (+ (eOccs x ss)) pu
  in oneEqZeroPlus (dOccs x ss) (eOccs x ss) h
```

## References


1. W. K. Wootters and W. H. Zurek, 'A single quantum cannot be cloned', Nature 299 (1982); D. Dieks, Physics Letters A 92 (1982).
2. A. K. Pati and S. L. Braunstein, 'Impossibility of deleting an unknown quantum state', Nature 404 (2000).
3. S. Abramsky, 'No-Cloning in Categorical Quantum Mechanics', in Semantic Techniques in Quantum Computation, Cambridge University Press (2010).
4. S. Abramsky, E. Haghverdi, and P. Scott, 'Geometry of Interaction and linear combinatory algebras', Mathematical Structures in Computer Science 12 (2002).
5. Project secret3, 'The Linearity Ladder' and 'The Entropy Budget of Multiplication', working papers, August 2026.